

Assignment Scheduling using Greedy and A* Search

1. Problem Description

This assignment focuses on scheduling a set of dependent tasks under resource constraints. Each assignment depends on one or more prerequisites, and cannot be executed until all dependencies are satisfied. Additionally, each assignment requires exactly one food item, and the cost of execution depends on the set of food items used per day. A group constraint limits the number of assignments that can be solved per day. The objective is to generate a schedule that minimizes total food cost while respecting all constraints.

2. Greedy Strategies

2.1 Greedy by Food Cost

In this strategy, assignments are prioritized based on the cost of their associated food items. At each step, the algorithm selects available assignments with lower food costs in order to minimize daily expenditure. This approach attempts to reduce the cumulative cost across all days. However, due to dependency constraints, the algorithm may be forced to select assignments with higher cost or mix multiple food types, which reduces its effectiveness in some cases.

2.2 Greedy by Earliest Deadline

This strategy schedules assignments as soon as they become available, following a topological ordering. By prioritizing early execution, it unlocks dependent tasks quickly and tends to minimize the total number of days required. However, it does not consider food cost explicitly, and therefore may result in higher total cost due to inefficient food combinations.

3. A* Search Approach

A* search is used to find an optimal schedule that minimizes total food cost. Each state represents a partially completed schedule, including completed assignments and accumulated cost. The algorithm explores different possible schedules while using a heuristic to guide the search efficiently.

g(n): Total cost incurred so far.

h(n): Estimated remaining cost based on minimum food cost and remaining assignments.

The heuristic is admissible as it underestimates the true remaining cost. This ensures that A* always finds an optimal solution.

4. Statistical Comparison

Input	Method	Days	Cost
input1	Greedy (Food)	9	22
input1	Greedy (Earliest)	8	22
input1	A*	11	22
input2	Greedy (Food)	8	18
input2	Greedy (Earliest)	8	18
input2	A*	10	18
input3	Greedy (Food)	8	21
input3	Greedy (Earliest)	8	21
input3	A*	10	19

5. Plots and Visualization

Figure 1: Cost Comparison across strategies.

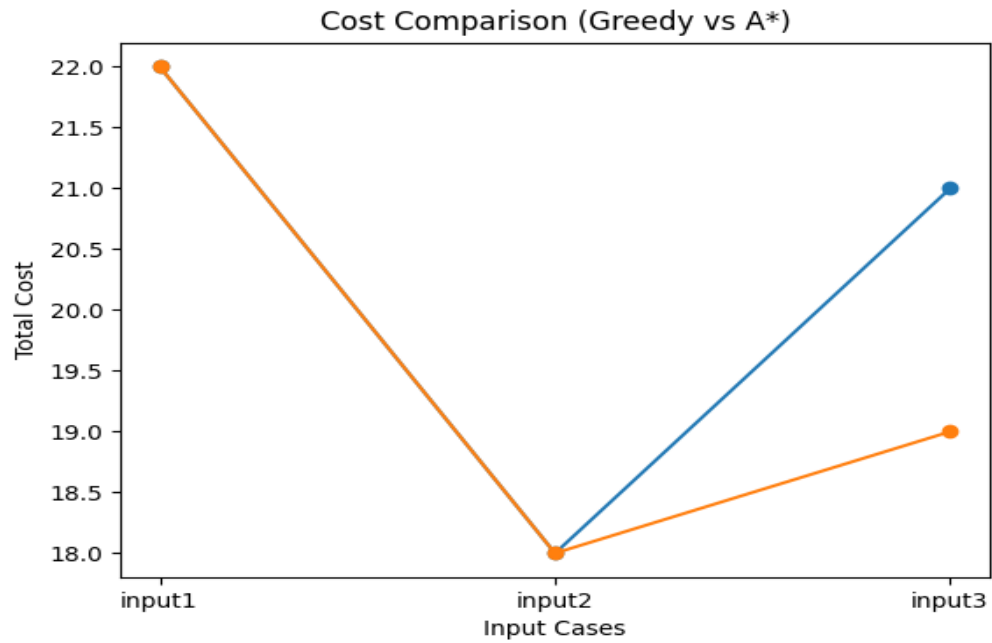
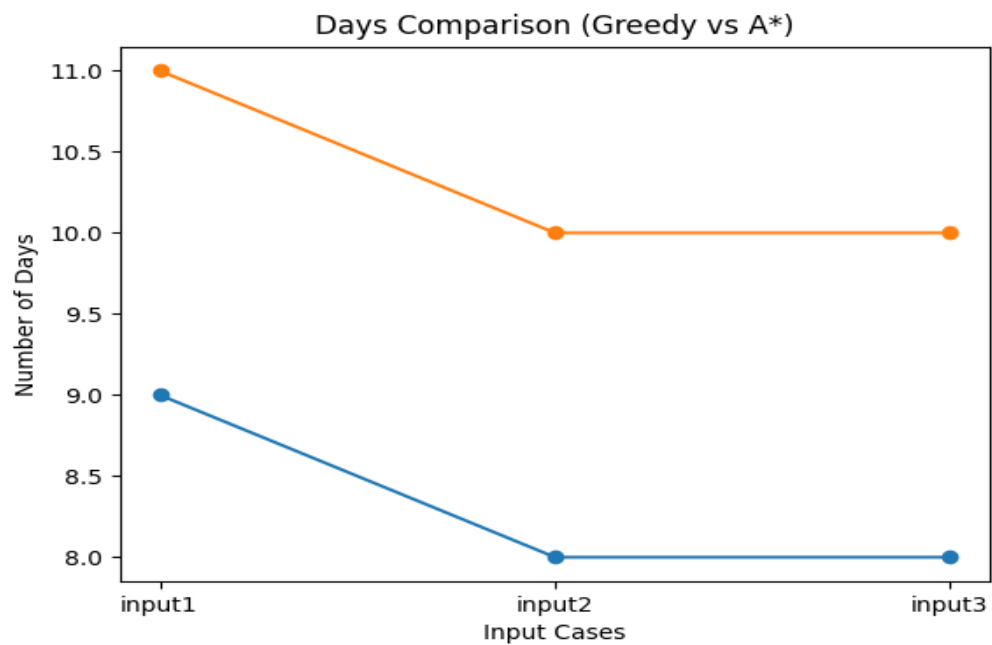


Figure 2: Days Comparison across strategies.



6. Analysis

The results demonstrate that greedy strategies are efficient and produce feasible schedules quickly. However, they do not always minimize total cost. In contrast, A* search guarantees optimal cost but may increase the number of days required. For example, in input3, A* reduces cost from 21 to 19, but increases the number of days from 8 to 10.

A key observation is the trade-off between cost and time. Greedy approaches tend to minimize time, while A* focuses on cost optimization. This highlights the importance of selecting the appropriate strategy based on the problem objective.

7. Conclusion

In conclusion, greedy algorithms are suitable for fast and practical scheduling, while A* search is ideal when optimal cost is required. The choice between these approaches depends on whether minimizing cost or minimizing execution time is the primary goal.